

Projet INFO-H-3000 : Planification Équitable des Tournois Round Robin avec Méta-heuristiques

Nicholas Birch de la Calle
Gabriel Badan
Milena Paques

Enseignant : Yves De Smet
Assistant : Jean Rosenfeld

Mai 2025

Table des matières

1	Introduction	3
1.1	Contexte	3
1.2	Objectifs du Projet	3
1.3	Structure du Rapport	3
2	Modélisation Mathématique	3
2.1	Définition du Problème	3
2.2	Variables de Décision	4
2.2.1	Variable Principale	4
2.2.2	Variables Auxiliaires pour l'Équité	4
2.3	Contraintes	4
2.3.1	Contraintes Fondamentales du Round Robin	4
2.3.2	Contraintes d'Équité	5
2.4	Métriques d'Équité	5
2.4.1	HomeStrength	5
2.4.2	Pénalité de Séquence	5
2.4.3	StdDevAdvantage (Proxy : MaxDev)	6
2.5	Fonction Objectif	6
2.6	Fonction Objectif Normalisée et Calibration des Poids (α, β)	6
3	Approche par Méthode Exacte	7
3.1	Formulation en Programmation Linéaire en Nombres Entiers (MILP)	7
3.2	Résultats pour Petites Instances	7
4	Approche par Méta-heuristique : Recuit Simulé (Simulated Annealing)	8
4.1	Principe et Implémentation	8
4.2	Réglage des Hyperparamètres du SA (α, β)	8
4.2.1	Recherche par Grille (Grid Search)	8
4.2.2	Optimisation avec Optuna	8
4.3	Showcase SA : Instance Très Grande $n = 1000$	8
5	Approche par Méta-heuristique : Recherche Tabou (Tabu Search)	9
5.1	Principe et Implémentation "Légère"	9
5.2	Résultats des Benchmarks TS	9

6	Analyse Comparative et Discussion	9
6.1	Comparaison MILP vs. Heuristiques (SA et TS)	9
6.2	Comparaison SA vs. TS	10
6.3	Impact du Réglage des Hyperparamètres (α, β) pour SA	10
6.4	Limites et Perspectives	10
7	Analyse de Complexité Empirique	10
8	Conclusion	11

1 Introduction

1.1 Contexte

La planification de tournois où chaque participant rencontre tous les autres une seule fois (tournoi "Round Robin" simple, ou SRR) est un problème classique en recherche opérationnelle et en gestion sportive. L'objectif de base est de s'assurer que toutes les rencontres requises aient lieu. Cependant, dans de nombreux contextes, l'équité du calendrier devient un enjeu majeur. Des avantages positionnels, comme jouer à domicile dans les sports d'équipe (football, basketball) ou jouer avec les pièces blanches aux échecs, peuvent influencer significativement l'issue des matchs et, par conséquent, le classement final du tournoi. Un calendrier qui attribue systématiquement les matchs à domicile contre les adversaires les plus faibles à une équipe, ou qui impose de longues séries de matchs à l'extérieur à une autre, peut être considéré comme injuste. Ce projet s'attaque au défi de construire des calendriers SRR qui non seulement respectent les contraintes de base, mais optimisent également l'équité selon plusieurs dimensions.

1.2 Objectifs du Projet

L'objectif principal de ce projet est de développer et d'évaluer des méthodes pour générer des calendriers de tournois SRR équitables pour un nombre n de joueurs (supposé pair). L'équité est définie à travers un ensemble de critères quantifiables :

- Équilibrage de l'avantage domicile/extérieur en fonction de la différence de force entre les adversaires (HomeStrength). Cette métrique évalue si les équipes les plus fortes ont tendance à recevoir les plus faibles, ou vice-versa.
- Minimisation des séries de matchs consécutifs joués à domicile ou à l'extérieur ("Pénalité de séquence").
- Homogénéité de la répartition des matchs domicile/extérieur entre tous les joueurs (Proxy : Max Deviation).

Pour atteindre cet objectif, nous avons exploré trois approches complémentaires :

1. Une méthode exacte basée sur la Programmation Linéaire en Nombres Entiers Mixtes (MILP) pour obtenir des solutions optimales sur de petites instances.
2. Une méta-heuristique (Recuit Simulé, SA) optimisée pour trouver des solutions de bonne qualité pour des instances de taille moyenne et grande dans un temps raisonnable.
3. Une seconde méta-heuristique (Recherche Tabou "légère", TS) pour comparer les approches heuristiques et explorer différentes stratégies de recherche.

1.3 Structure du Rapport

Ce rapport détaille d'abord la modélisation mathématique du problème (Section 2), incluant les variables, contraintes, et métriques d'équité. Ensuite, nous présentons l'approche exacte (Section 3) et ses résultats. Les approches par méta-heuristiques, Recuit Simulé (Section 4) et Recherche Tabou (Section 5), sont décrites ensuite, avec leurs implémentations, optimisations, et résultats de benchmarks, incluant le réglage des hyperparamètres. Une analyse comparative des trois approches et une discussion des résultats sont fournies (Section 6), suivies d'une analyse de complexité (Section 7) et d'une conclusion (Section 8).

2 Modélisation Mathématique

2.1 Définition du Problème

Le problème consiste à générer un calendrier pour un tournoi Round Robin simple (SRR) impliquant n joueurs, où n est pair. Les joueurs sont numérotés de 1 à n , et cet indice représente

également leur classement de force (1 étant le plus fort, n le plus faible). Le tournoi se déroule sur $n-1$ rondes (numérotées de 0 à $n-2$). Dans chaque ronde, $n/2$ matchs sont joués simultanément. Chaque paire de joueurs $\{i, j\}$ doit s'affronter exactement une fois au cours du tournoi. Pour chaque match, l'un des joueurs est désigné comme jouant "à domicile" (H) et l'autre "à l'extérieur" (A). L'objectif est de produire un calendrier complet qui minimise une fonction d'équité combinant plusieurs critères.

Entrées :

- n : Le nombre de joueurs (pair).

Sortie :

- Un calendrier complet spécifiant pour chaque ronde $r \in \{0, \dots, n-2\}$ et chaque match (i, j) qui joue à domicile (avec $i, j \in \{1, \dots, n\}$).

2.2 Variables de Décision

2.2.1 Variable Principale

La décision fondamentale pour chaque match potentiel est de savoir qui joue à domicile. Nous utilisons une variable binaire principale pour cela :

$$x_{i,j,r} = \begin{cases} 1 & \text{si le joueur } i \text{ joue à domicile contre le joueur } j \text{ lors de la ronde } r \\ 0 & \text{sinon} \end{cases} \quad (1)$$

où $P = \{1, \dots, n\}$ est l'ensemble des joueurs (avec $i \neq j$) et $R = \{0, \dots, n-2\}$ représente l'ensemble des rondes. L'indice du joueur correspond à son classement (1 étant le plus fort).

2.2.2 Variables Auxiliaires pour l'Équité

Pour modéliser les métriques d'équité, nous introduisons les variables auxiliaires suivantes :

- $is_home_{i,r}$: Variable binaire valant 1 si le joueur $i \in P$ joue à domicile lors de la ronde $r \in R$.
- $pen_seq_{i,r}$: Variable binaire valant 1 si le joueur $i \in P$ subit une "pénalité de séquence" (deux matchs consécutifs à domicile ou à l'extérieur) entre les rondes r et $r+1$. Définie pour $r \in R' = \{0, \dots, n-3\}$.
- H_i : Variable entière représentant le nombre total de matchs joués à domicile par le joueur $i \in P$.
- $MaxDev$: Variable continue représentant l'écart absolu maximal entre le nombre de matchs à domicile d'un joueur (H_i) et la moyenne idéale $\bar{H} = \frac{n-1}{2}$.

2.3 Contraintes

2.3.1 Contraintes Fondamentales du Round Robin

- **Un match par joueur par ronde** : Chaque joueur doit jouer exactement un match lors de chaque ronde.

$$\sum_{j \in P, j \neq i} x_{i,j,r} + \sum_{j \in P, j \neq i} x_{j,i,r} = 1 \quad \forall i \in P, \forall r \in R \quad (2)$$

- **Chaque paire se rencontre une fois** : Chaque paire de joueurs $\{i, j\}$ doit s'affronter exactement une fois au cours du tournoi.

$$\sum_{r \in R} (x_{i,j,r} + x_{j,i,r}) = 1 \quad \forall i, j \in P, i < j \quad (3)$$

2.3.2 Contraintes d'Équité

Ces contraintes lient les variables auxiliaires aux variables de décision $x_{i,j,r}$.

— **Lien $is_home_{i,r}$** : Détermine si un joueur est à domicile.

$$is_home_{i,r} = \sum_{j \in P, j \neq i} x_{i,j,r} \quad \forall i \in P, \forall r \in R \quad (4)$$

— **Lien $pen_seq_{i,r}$ (Pénalités Séquence)** : Une pénalité se produit si $is_home_{i,r} = is_home_{i,r+1}$. Ceci est linéarisé comme suit :

$$pen_seq_{i,r} \geq is_home_{i,r} + is_home_{i,r+1} - 1 \quad \forall i \in P, \forall r \in R' \quad (5)$$

$$pen_seq_{i,r} \geq (1 - is_home_{i,r}) + (1 - is_home_{i,r+1}) - 1 \quad \forall i \in P, \forall r \in R' \quad (6)$$

où $R' = \{0, \dots, n-3\}$.

— **Lien H_i (Total Domicile)** : Calcule le nombre total de matchs à domicile.

$$H_i = \sum_{r \in R} is_home_{i,r} \quad \forall i \in P \quad (7)$$

— **Lien $MaxDev$ (Écart Max Domicile)** : Calcule l'écart maximal par rapport à la moyenne $\bar{H} = \frac{n-1}{2}$.

$$MaxDev \geq H_i - \bar{H} \quad \forall i \in P \quad (8)$$

$$MaxDev \geq \bar{H} - H_i \quad \forall i \in P \quad (9)$$

2.4 Métriques d'Équité

Les objectifs d'équité sont quantifiés par les métriques suivantes, que nous cherchons à minimiser (ou, dans le cas de HomeStrength, à amener proche de zéro).

2.4.1 HomeStrength

Cette métrique évalue l'équilibre de la force des adversaires rencontrés par rapport au lieu du match (domicile/extérieur). Elle est définie comme la somme, sur tous les matchs, de la différence de rang entre l'adversaire et le joueur à domicile. En supposant que l'indice du joueur k (de 1 à n) représente son classement (1 étant le plus fort), la métrique est :

$$HomeStrength = \sum_{i \in P} \sum_{j \in P, j \neq i} \sum_{r \in R} (j - i) \cdot x_{i,j,r} \quad (10)$$

où $x_{i,j,r} = 1$ si le joueur i (domicile) joue contre le joueur j (extérieur) en ronde r . Une valeur positive indique une tendance pour les joueurs mieux classés (petit indice i) à recevoir des joueurs moins bien classés (grand indice j), ou pour les joueurs moins bien classés (grand i) à recevoir des joueurs encore moins bien classés (très grand j). Inversement, une valeur négative indique que les joueurs moins bien classés tendent à recevoir les mieux classés. L'objectif est d'avoir une valeur proche de zéro, indiquant un équilibre global. La normalisation de cette métrique (voir Section 2.6) la ramène à un intervalle $[-1, 1]$.

2.4.2 Pénalité de Séquence

Une "pénalité de séquence" est définie comme deux matchs consécutifs joués soit à domicile (H-H), soit à l'extérieur (A-A). Le nombre total de pénalités est la somme des variables auxiliaires $pen_seq_{i,r}$:

$$TotalPenalitesSequence = \sum_{i \in P} \sum_{r \in R'} pen_seq_{i,r} \quad (11)$$

où $R' = \{0, \dots, n-3\}$. Minimiser cette somme favorise une alternance H/A plus régulière.

2.4.3 StdDevAdvantage (Proxy : MaxDev)

Pour assurer une répartition homogène du nombre de matchs joués à domicile entre les joueurs, nous cherchons à minimiser la variance de H_i . La variance étant une fonction quadratique ($\sum (H_i - \bar{H})^2 / n$), elle ne peut être directement utilisée dans un objectif linéaire. Nous utilisons donc une approximation linéaire courante : minimiser l'écart absolu maximal par rapport à la moyenne $\bar{H} = \frac{n-1}{2}$. Cette valeur, capturée par la variable *MaxDev*, sert de proxy pour l'homogénéité de la répartition des matchs à domicile.

$$\text{MaxDeviation} = \text{MaxDev} \quad (12)$$

2.5 Fonction Objectif

Initialement, nous combinons les trois métriques d'équité principales en une fonction objectif pondérée unique. Pour HomeStrength, l'objectif est d'avoir une valeur proche de zéro. Pour les autres, c'est la minimisation.

$$\text{Objectif} = f(\text{HomeStrength}, \text{TotalPenalitesSequence}, \text{MaxDeviation}) \quad (13)$$

La fonction exacte, incluant la pondération et la normalisation, est détaillée ci-dessous.

2.6 Fonction Objectif Normalisée et Calibration des Poids (α, β)

Pour comparer et pondérer équitablement les différentes métriques, nous utilisons des fonctions de normalisation définies dans `src/metrics.py`. La métrique HomeStrength (Eq. 10) est normalisée pour viser un intervalle de $[-1, 1]$. Les autres métriques sont normalisées pour viser $[0, 1]$. Les dénominateurs utilisés pour la normalisation pour un nombre n de joueurs sont :

- Pour HomeStrength : $S_{max} = \frac{n(n-1)(n+1)}{6}$. C'est la valeur absolue maximale théorique de HomeStrength.
- Pour TotalPenalitesSequence : $D_{PS} = n(n-2)$.
- Pour MaxDeviation : $D_{MD} = \frac{n-1}{2}$.

La fonction objectif normalisée que nous cherchons à minimiser est :

$$\min \quad Z_{norm} = \frac{|\text{HomeStrength}|}{S_{max}} + \alpha \cdot \frac{\text{TotalPenalitesSequence}}{D_{PS}} + \beta \cdot \frac{\text{MaxDeviation}}{D_{MD}} \quad (14)$$

Soit $Z_{norm} = |\text{HS}_{norm}| + \alpha \cdot \text{PS}_{norm} + \beta \cdot \text{MD}_{norm}$. L'utilisation de la valeur absolue pour HS_{norm} assure que nous minimisons l'amplitude de tout déséquilibre de force, qu'il favorise les forts ou les faibles. Dans le code MILP, cela est géré en minimisant HS_{norm} directement, car le solveur tendra vers zéro pour un terme non-carré dans une minimisation. Pour le SA, la valeur absolue est conceptuellement ce que nous visons, bien que le score puisse être calculé avec HS_{norm} signé si l'objectif est de le pousser vers zéro. **Note importante** : La discussion sur la calibration des poids α et β ci-dessous, ainsi que les valeurs numériques spécifiques de HS_{norm} mentionnées, sont basées sur une *ancienne définition* de HomeStrength et sa normalisation. Une nouvelle calibration serait nécessaire avec la métrique HomeStrength actuelle (Eq. 10) et la normalisation par S_{max} .

Le choix des poids α et β est crucial. Pour les fixer (avec l'ancienne métrique), nous avons effectué une calibration sur $n = 6$. Nous avons exécuté le script `src/exact_model.py` avec une grille de valeurs pour α et β allant de 0.0 à 6.0, par pas de 0.2, pour $n = 6$. Les résultats ont été sauvegardés dans `reports/alpha_beta_calibration_n6_extended_range.csv`.

Après filtrage pour ne conserver que les solutions non-dominées (basées sur l'ancienne HS_{norm}), la frontière de Pareto pour $n = 6$ (Figure 1) a guidé notre choix.

L'analyse de cette frontière de Pareto (Figure 1, basée sur l'ancienne métrique HS_{norm}) et des données du fichier CSV `alpha_beta_calibration_n6_extended_range.csv` a guidé notre choix initial pour α et β . L'objectif était de minimiser fortement PS_{norm} et MD_{norm} .

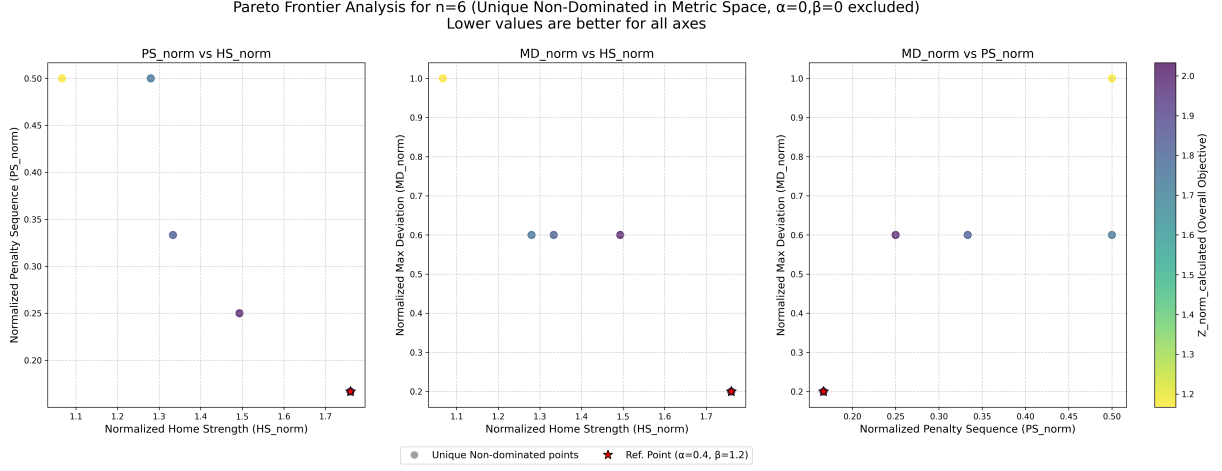


FIGURE 1 – Frontière de Pareto pour $n = 6$ issue de la calibration étendue (données de `alpha_beta_calibration_n6_extended_range.csv`). Les graphes montrent les projections 2D des 5 uniques solutions non-dominées trouvées. Le point rouge étoilé correspond aux métriques obtenues avec $\alpha = 0.4, \beta = 1.2$, qui sont les plus proches de notre cible de configuration. Les valeurs plus faibles sont meilleures pour tous les axes.

Avec l’ancienne métrique, les valeurs cibles étaient $PS_{norm} \approx 0.167$ et $MD_{norm} \approx 0.20$. La combinaison ($\alpha = 0.4, \beta = 1.2$) atteignait ces valeurs, avec l’ancienne $HS_{norm} \approx 1.76$.

Les valeurs $\alpha = 0.5$ et $\beta = 1.0$ ont été retenues pour `src/config.py`. Ces valeurs sont basées sur l’ancienne définition de HomeStrength et sa normalisation. **Une nouvelle calibration est nécessaire pour déterminer les poids optimaux avec la métrique HomeStrength actuelle (Eq. 10) et la fonction objectif (Eq. 14) qui utilise $|HS_{norm}|$.**

3 Approche par Méthode Exacte

3.1 Formulation en Programmation Linéaire en Nombres Entiers (MILP)

Le modèle décrit ci-dessus correspond à un Programme Linéaire en Nombres Entiers Mixtes (MILP). Nous avons implémenté ce modèle en utilisant la librairie Python PuLP, qui permet de décrire le problème de manière symbolique et d’appeler un solveur externe comme CBC. L’implémentation se trouve dans `src/exact_model.py`. Le script a été modifié pour accepter une limite de temps via la ligne de commande.

3.2 Résultats pour Petites Instances

Nous avons exécuté le modèle exact pour $n = 4, 6$, et 8 joueurs. Pour $n = 8$ (avec l’ancienne métrique HomeStrength), avec $\alpha = 1, \beta = 1$ et une limite de temps de 300s, le solveur a trouvé la solution optimale en 0.1s. Le calendrier obtenu présentait une structure où le joueur le plus fort (indice 1) jouait tous ses matchs à l’extérieur, et le plus faible (indice 8) tous à domicile. Cette configuration, bien qu’optimale pour la fonction objectif donnée (ancienne version), interroge sur la perception d’équité. Pour $n > 8$, l’approche MILP devient rapidement impraticable.

4 Approche par Méta-heuristique : Recuit Simulé (Simulated Annealing)

4.1 Principe et Implémentation

Le Recuit Simulé (SA) a été implémenté (dans `src/sa_solver.py`) avec les optimisations suivantes :

- **Mise à Jour Incrémentale des Métriques avec Numba** : Accélère significativement l'évaluation des voisins.
- **Cache Numba Partagé** : `NUMBA_CACHE_DIR` configuré pour réduire les temps de redémarrage.
- **Parallélisation avec ProcessPoolExecutor Partagé** : Pour les exécutions multiples, afin de réduire le surcoût.
- **Gestion des Threads Numba** : `os.environ["NUMBA_THREADING_LAYER"] = "omp"` et `numba.set_num_threads()` sont utilisés pour contrôler le nombre de threads par worker Numba.

4.2 Réglage des Hyperparamètres du SA (α, β)

4.2.1 Recherche par Grille (Grid Search)

Effectuée pour $\alpha, \beta \in \{0.25, 1.0, 4.0\}$ sur $n = 30$ (2 runs) et $n = 200$ (11 chaînes). Les heatmaps générées par `benchmark_sa.py` sont présentées en Figure 2. Un exemple de planning pour $n = 30$ issu de ces tests est disponible en Annexe ??.

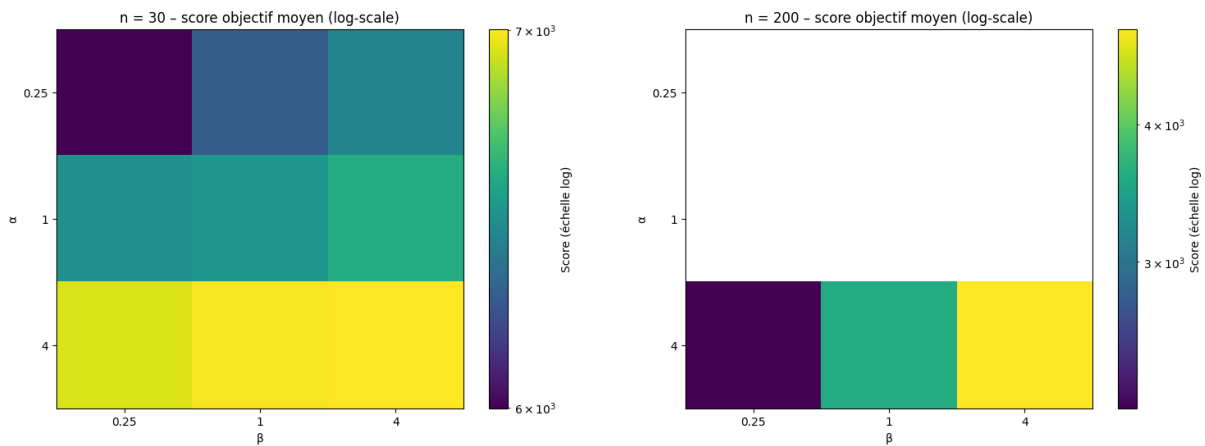


FIGURE 2 – Cartes de chaleur (échelle log) des scores objectifs moyens du SA en fonction de α et β pour $n = 30$ (gauche) et $n = 200$ (droite), générées par `benchmark_sa.py`.

4.2.2 Optimisation avec Optuna

Pour $n = 200$ (50 essais, minimisation du score normalisé), Optuna a trouvé $\alpha \approx 0.1196$, $\beta \approx 0.1145$ (score normalisé 1.1285). Ces valeurs diffèrent de la grille et des suggestions initiales du guide ($\alpha = 0.5, \beta = 2.0$). (Voir `sa_alpha_beta_tuning_n200.db`).

4.3 Showcase SA : Instance Très Grande $n = 1000$

Avec $\alpha = 0.5, \beta = 2.0$, 6000 itérations/chaîne, 11 cœurs. Temps d'exécution : ~ 5.32 s. Score objectif : $\sim 250\,686\,943$. Le planning est dans `reports/planning_1000.csv`. La distribution des matches à domicile pour cette instance est montrée en Figure 3.

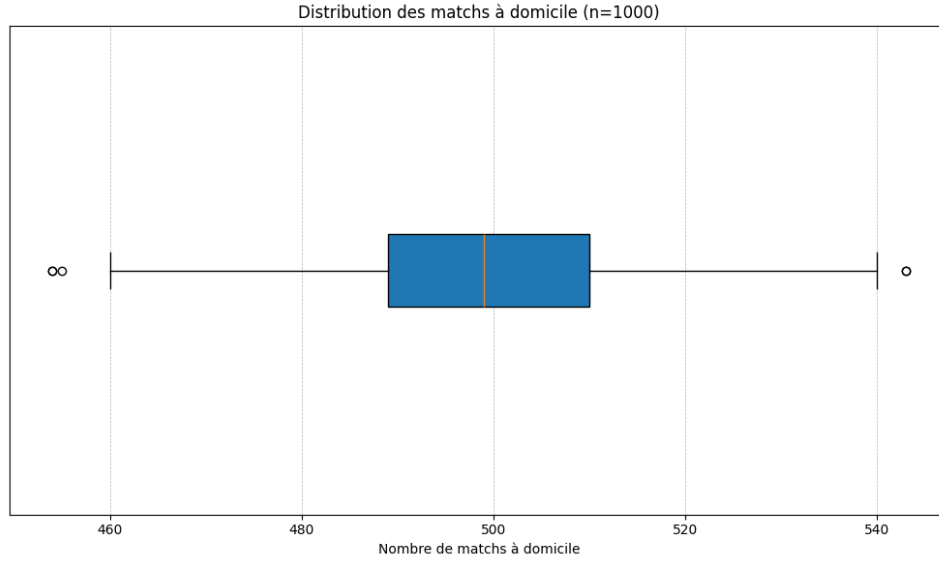


FIGURE 3 – Distribution des matchs à domicile par joueur pour $n = 1000$ (SA Showcase).

TABLE 1 – Résultats SA pour $n = 1000$ (11 chaînes, $6k$ iters/chaîne, $\alpha = 0.5, \beta = 2.0$)

n	Objectif	Temps (s)
1000	250,686,943	5.32

5 Approche par Méta-heuristique : Recherche Tabou (Tabu Search)

5.1 Principe et Implémentation "Légère"

Une version "légère" de TS a été implémentée (`tabu_search_solver` dans `src/sa_solver.py`).

- Voisinage : Sélection des k meilleurs flips H/A parmi $k \times 5$ générés, avec recalcul complet des métriques (non incrémental).
- Liste Tabou : `collections.deque`, attribut (`round_idx`, `match_idx`).
- Tenure : Adaptative, $\max(10, \text{int}(0.1 \cdot n))$.
- Critère d'Aspiration : Accepte un mouvement tabou s'il améliore le meilleur score global.

5.2 Résultats des Benchmarks TS

Tests sur $n = 30, 200$ ($\alpha = 0.5, \beta = 2.0$, 200 itérations, $k = 5$). Un `RuntimeWarning` d'overflow pour `pen_seq` a été observé, causant des valeurs négatives. Cela indique une différence de calcul par rapport au SA et affecte la comparabilité des scores.

6 Analyse Comparative et Discussion

6.1 Comparaison MILP vs. Heuristiques (SA et TS)

Le MILP est optimal pour $n \leq 8$ mais non échelonnable. Le SA optimisé est très rapide ($n = 1000$ en $\sim 5s$). Le TS actuel est plus lent dû au recalcul des métriques. La solution MILP pour $n = 8$ (joueur fort toujours à l'extérieur, faible toujours à domicile) soulève des questions sur l'équité perçue vs. mathématique.

6.2 Comparaison SA vs. TS

Comparaison difficile à cause de paramètres et d'implémentations différents (surtout calcul de `pen_seq` et optimisation SA vs. TS). Pour $n = 200$, SA (avec $\alpha = 0.25, \beta = 0.25$) score $\sim 1.96 \times 10^6$ (issu du CSV grid search) ; TS ($\alpha = 0.5, \beta = 2.0$) score $\sim 1.98 \times 10^6$. Le SA est plus rapide.

6.3 Impact du Réglage des Hyperparamètres (α, β) pour SA

La recherche par grille et Optuna pour SA ($n = 200$) suggèrent que des α, β plus faibles (ex : $\sim 0.1 - 0.25$) donnent de meilleurs scores bruts/normalisés que les valeurs initiales du guide ($\alpha = 0.5, \beta = 2.0$). Le tuning est crucial.

6.4 Limites et Perspectives

- **MILP** : Non échelonnable.
- **SA** : Ne garantit pas l'optimum. Paramétrage sensible. Voisinage simple.
- **TS** : Implémentation non optimisée. Problème de calcul `pen_seq` à corriger.
- **Métriques** : Explorer d'autres métriques.
- **Travaux Futurs** :
 - Comparaison SA/TS rigoureuse (paramètres/optimisations alignés).
 - Intégrer contraintes plus complexes.

7 Analyse de Complexité Empirique

L'analyse se concentre sur le SA optimisé. La Figure 4 illustre la scalabilité du temps d'exécution moyen en fonction de n . La complexité par chaîne SA est $O(N_{iter} \cdot n)$ en moyenne, $O(N_{iter} \cdot n^2)$ au pire (MaxDev). La parallélisation est efficace ($n = 1000$ en $\sim 5s$). Le TS actuel est $O(k \cdot N_{iter} \cdot n^2)$ par chaîne, moins compétitif sans optimisation incrémentale.

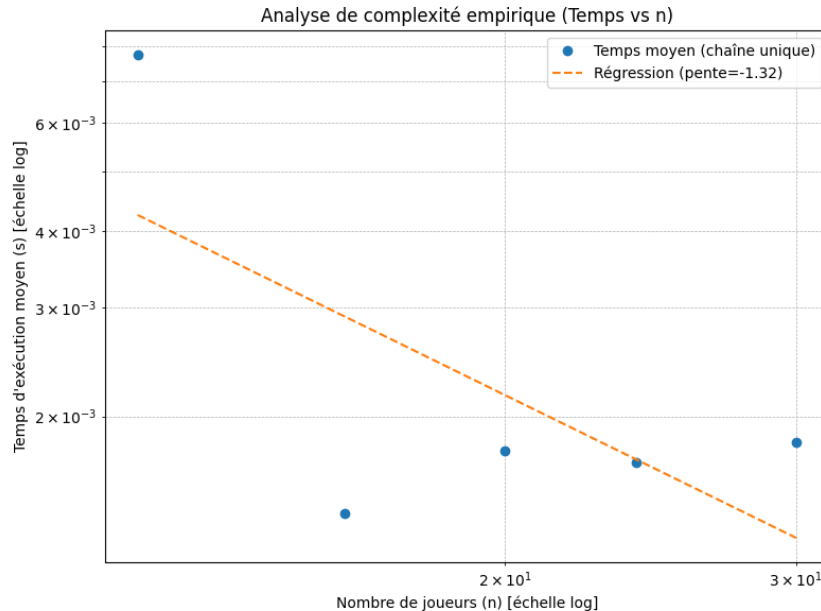


FIGURE 4 – Analyse de complexité empirique du SA : Temps d'exécution moyen vs. n (échelle log-log).

8 Conclusion

Ce projet a exploré la génération de calendriers SRR équitables via MILP, SA et TS. Le MILP est optimal pour $n \leq 8$. Le SA optimisé est très performant et échelonnable ($n = 1000$ en $\sim 5s$), avec des résultats sensibles au tuning des hyperparamètres α, β (Optuna suggérant des valeurs plus faibles que celles initialement prévues). Le TS "léger" a été implémenté. Des problèmes de calcul dans TS et son manque d'optimisation limitent la comparaison directe actuelle. Les travaux futurs devraient se concentrer sur l'amélioration du TS et une comparaison plus rigoureuse des heuristiques. Les résultats du tuning d'hyperparamètres sont des contributions clés de cette phase du projet.